SERIE · 4 FORMAS DE ORQUESTAR AGENTES DE IA

Secuencial | Paralela | Jerárquica | ...

Orquestación paralela de agentes de IA

Guía práctica para implementarla y monitorizarla en Claude Code, con plantillas listas para copiar.

@chanti.dev · Edición octubre 2026

Basada en la documentación oficial de Claude Code (subagentes y hooks).

CONTENIDO

Índice

01	Qué es la orquestación paralela	definición, flujo, diferencia con la secuencial
02	Para qué sirve y cuándo usarla	casos de uso y test de independencia
03	Cómo funciona en Claude Code	roles, contexto, límites
04	Implementación paso a paso	de cero a tu primera ejecución
05	Ejemplo completo: revisión de código en paralelo	3 agentes + agregación
06	Cómo monitorizarla	panel, /tasks, transcripts, hooks, logs
07	Errores frecuentes y cómo resolverlos	diagnóstico rápido
08	Buenas prácticas y costes	modelos, contexto, límites de uso
09	Plantillas	10 plantillas listas para copiar o rellenar
10	Chuleta de referencia rápida	todo en una página

CÓMO USAR ESTA GUÍA

Si solo quieres ponerla en marcha, ve directamente a la **Parte 04** y a las **Plantillas**. Si quieres entender por qué funciona así (y qué hacer cuando no funciona), lee en orden. Los nombres de archivos, campos y comandos están verificados con la documentación de Claude Code de octubre de 2026; si usas una versión antigua, actualiza con `claude update`.

01 · CONCEPTO

Qué es la orquestación paralela

Varios agentes trabajan a la vez sobre partes distintas de una misma tarea y, al final, se unen sus resultados.



El flujo, paso a paso

1. **El usuario envía una tarea.** Por ejemplo: «revisa esta rama antes de hacer merge».
2. **El orquestador la recibe, la divide en partes y la reparte.** Decide qué subtareas hay y qué agente se encarga de cada una.
3. **Los agentes arrancan a la vez.** Cada uno con su subtarea. Ninguno espera a otro y ninguno ve lo que hacen los demás.
4. **El agregador espera a que terminen todos.** Recoge los resultados y los une en una sola respuesta.
5. **La respuesta vuelve al usuario.** Una sola, coherente, sin duplicados.

01 · CONCEPTO

Paralela frente a secuencial

En la orquestación **secuencial** la salida de un agente es la entrada del siguiente: B no puede empezar hasta que A termina. En la **paralela** los tres agentes arrancan en el mismo momento porque sus subtareas no dependen unas de otras.

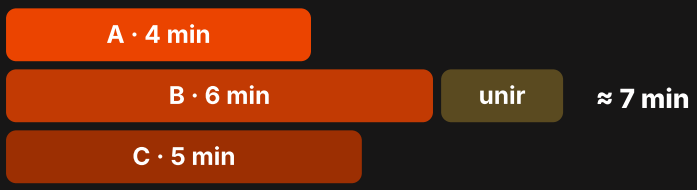
SECUENCIAL

cada agente espera al anterior · tiempo = A + B + C



PARALELA

todos arrancan a la vez · tiempo ≈ el más lento + agregación



El tiempo total de la secuencial es la **suma** de los agentes. El de la paralela se acerca al del **agente más lento**, más lo que tarde la agregación.

	Secuencial	Paralela
Dependencias	Cada paso necesita el anterior	Las subtareas son independientes
Tiempo	Suma de todos los pasos	≈ el paso más lento + agregación
Contexto que pasa	Resultado de A → entrada de B	Cada agente recibe solo su parte
Pieza extra	—	Un agregador que une resultados
Riesgo típico	Un fallo temprano arrastra al resto	Resultados que se pisan o se contradicen
Coste	Similar en tokens	Similar en tokens, más consumo simultáneo

IDEA CLAVE

Paralelizar no reduce el trabajo total: reduce el tiempo de espera. Los tokens que gastan los agentes son prácticamente los mismos; lo que cambia es que se gastan a la vez.

02 · UTILIDAD

Para qué sirve y cuándo usarla

Úsala cuando las subtareas no dependen unas de otras

Esa es la única condición imprescindible. Si para hacer la parte B necesitas el resultado de la parte A, no es paralela: es secuencial (o una mezcla).

Caso de uso	Cómo se divide
Revisión de código	Un agente por especialidad: seguridad, rendimiento, tests, estilo.
Investigación de un repositorio	Un agente por módulo: autenticación, base de datos, API.
Investigación web	Un agente por fuente o pregunta: documentación oficial, foros, comparativas.
Migraciones mecánicas	Un agente por carpeta, cada uno en su propio worktree de git.
Auditorías	SEO técnico, accesibilidad y rendimiento de una web, cada uno por separado.
Contenido	A partir de un mismo guion: un agente para el caption, otro para la descripción de YouTube, otro para el hilo de X.

Test de independencia (antes de paralelizar)

- [ ] ¿Cada subtarea se puede explicar sin mencionar el resultado de otra?
- [ ] ¿Los agentes trabajan sobre archivos distintos, o solo leen?
- [ ] ¿El orden en que terminen da igual?
- [ ] ¿Puedes definir un formato de salida común para unir los resultados?
- [ ] ¿La tarea es lo bastante grande para compensar arrancar varios agentes?

Si respondes «sí» a todas, la paralela encaja. Si alguna es «no», revisa la división o combina patrones (por ejemplo: un paso secuencial de preparación y después un bloque en paralelo).

NO LA USES SI...

- Varios agentes tienen que editar los mismos archivos.
- La tarea es pequeña y la resuelves en una conversación.
- Necesitas ida y vuelta constante con el usuario.
- Una subtarea necesita el resultado de otra.

VENTAJAS

- Menos tiempo de espera.
- Cada agente tiene su propio contexto limpio.
- Especialización: prompt, herramientas y modelo por agente.
- La conversación principal solo recibe resúmenes.

03 · FUNCIONAMIENTO

Cómo funciona en Claude Code

En Claude Code, **tu conversación principal hace de orquestador y de agregador**. Los agentes son **subagentes**: archivos Markdown con frontmatter YAML que defines en `.claude/agents/`.

Rol del patrón	En Claude Code	Qué hace
Usuario	Tú, en el prompt	Describe la tarea y pide ejecutarla en paralelo.
Orquestador	Conversación principal	Divide la tarea y lanza varias llamadas a la herramienta `Agent` en el mismo turno.
Agentes	Subagentes de `.claude/agents/`	Trabajan a la vez, cada uno en su propia ventana de contexto.
Agregador	Conversación principal	Recibe una notificación por cada subagente que termina y, con todos, redacta la respuesta final.

Qué recibe cada subagente al arrancar

- **Su propio system prompt**: el cuerpo de su archivo `.md` (no el system prompt de Claude Code).
- **El mensaje de delegación** que escribe el orquestador con la subtarea.
- **Tus archivos CLAUDE.md** y una instantánea del **git status** (salvo que pongas `omitClaudeMd: true`).
- Las **skills** que indiques en su campo `skills`.

> **LO QUE NO RECIBE**
>
> El historial de tu conversación. Un subagente no sabe lo que habéis hablado antes: solo conoce lo que el orquestador le pasa en la delegación. Por eso la calidad de la delegación decide la calidad del resultado.

Cómo se ejecutan

- Los subagentes que Claude lanza desde la conversación principal **arrancan en segundo plano por defecto**. Puedes seguir escribiendo mientras trabajan.
- Cuando uno termina, su resultado llega a la conversación principal como una **notificación de finalización**. Claude espera a tenerlas antes de dar el resultado; si le preguntas antes, te dirá que siguen en marcha.
- Si un subagente necesita un permiso, el aviso aparece en tu sesión principal indicando **qué subagente lo pide**. Esc deniega solo esa llamada, sin detener al agente.
- Los subagentes en segundo plano usan un conjunto reducido de herramientas integradas (Read, Grep, Glob, Bash, Edit, Write, WebFetch, WebSearch, Skill, entre otras) y todas las herramientas MCP.

03 · FUNCIONAMIENTO

Límites que debes conocer

Límite	Valor por defecto	Cómo cambiarlo
Subagentes en marcha a la vez	20	`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`. Al llegar al límite, el siguiente lanzamiento falla con `Concurrent subagent limit reached`.
Profundidad de anidación	3 capas bajo la conversación principal	`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`. Pon `1` para que ningún subagente lance otros.
Turnos por subagente	Sin tope propio	Campo `maxTurns`. Al alcanzarlo, devuelve su salida marcada como parcial y se puede reanudar.
Descripciones de agentes	Aviso a partir de 15.000 tokens sumados	Acorta los `description` y lleva el detalle al cuerpo del archivo.
Límites de uso del plan	Compartidos	Cada subagente hace sus propias peticiones y cuentan contra los mismos límites que tu conversación.

Dónde viven los subagentes (y cuál gana)

Ubicación	Alcance	Prioridad
Ajustes gestionados de la organización	Toda la organización	1 (máxima)
Flag `--agents` (JSON)	Solo esa sesión	2
`.claude/agents/`	Proyecto actual (versiónalo en git)	3
`~/.claude/agents/`	Todos tus proyectos	4
Carpeta `agents/` de un plugin	Donde esté activo el plugin	5 (mínima)

Si dos subagentes tienen el mismo `name`, gana el de mayor prioridad. Las carpetas se leen de forma recursiva, así que puedes organizarlas en subcarpetas (`agents/review/`, `agents/research/`): la identidad la da solo el campo `name`.

PARALELO EN EL MISMO REPOSITORIO

Si varios agentes van a **escribir** archivos, dales `isolation: worktree`: cada uno trabaja en una copia aislada del repositorio (un worktree de git) y no se pisan. Si no hacen cambios, el worktree se limpia solo. Si solo leen, limita sus herramientas a `Read, Grep, Glob`.

04 · IMPLEMENTACIÓN

Implementación paso a paso

Del proyecto vacío a tu primera ejecución en paralelo.

0 Prepara el entorno

- Claude Code instalado y actualizado (`claude update`).
- Una carpeta de proyecto, idealmente un repositorio git (necesario si vas a usar worktrees).
- `jq` instalado si quieres el sistema de logs de la Parte 06.

1 Diseña la división antes de escribir nada

Rellena la **Plantilla 1**. Para cada subtarea decide: qué entra, qué sale, qué herramientas necesita y qué archivos toca. Si dos subtareas tocan los mismos archivos, fusiónalas o usa worktrees.

2 Crea la estructura de carpetas

terminal

```
mkdir -p .claude/agents .claude/hooks .claude/logs
```

estructura recomendada

```
mi-proyecto/
├── .claude/
│   ├── agents/                    ← un .md por subagente
│   │   ├── revisor-seguridad.md
│   │   ├── revisor-rendimiento.md
│   │   └── revisor-tests.md
│   ├── skills/
│   │   └── paralelo/SKILL.md      ← opcional: comando /paralelo
│   ├── hooks/
│   │   └── log-subagentes.sh      ← opcional: monitorización
│   └── settings.json              ← hooks y variables
├── CLAUDE.md                      ← reglas que heredan los subagentes
└── src/ ...
```

3 Escribe un archivo por subagente

Cada archivo tiene dos partes: el **frontmatter** entre `---` (configuración) y el **cuerpo** (su system prompt). Solo `name` y `description` son obligatorios. Usa la **Plantilla 2** como base.

También puedes pedírselo a Claude: *«Crea en .claude/agents/ un subagente revisor-seguridad de solo lectura que use Sonnet y devuelva este formato...»*. Después abre el archivo y comprueba el frontmatter.

04 · IMPLEMENTACIÓN

Campos del frontmatter

Los nombres de campo van en camelCase y deben coincidir exactamente: Claude Code **ignora sin avisar** un campo que no reconoce.

Campo	Para qué sirve en paralelo
`name` *	Identificador único (minúsculas y guiones). No puede contener `:`. Los hooks lo reciben como `agent_type`.
`description` *	Cuándo delegar en él. Añade «Úsalo en paralelo con X e Y» para que el orquestador los agrupe.
`tools`	Lista blanca de herramientas. Agentes de solo lectura: `Read, Grep, Glob`. Si lo omites, hereda todas.
`disallowedTools`	Lista negra. Ej.: `Write, Edit` para que no escriba.
`model`	`haiku`, `sonnet`, `opus`, `fable`, un ID completo o `inherit`. Tareas mecánicas en Haiku abaratan mucho el paralelo.
`effort`	`low` · `medium` · `high` · `xhigh` · `max`, según el modelo.
`maxTurns`	Tope de turnos. Evita que un agente se alargue y retrase la agregación.
`isolation`	`worktree`: copia aislada del repo. Imprescindible si varios agentes escriben.
`permissionMode`	`default`, `acceptEdits`, `auto`, `dontAsk`, `plan`… Si tu sesión está en `acceptEdits`, `auto` o `bypassPermissions`, el subagente usa el modo de la sesión.
`background`	`true` lo mantiene en segundo plano incluso si Claude intenta esperarlo.
`skills`	Skills que se cargan completas en su contexto al arrancar.
`mcpServers`	Servidores MCP solo para este agente (no ocupan contexto en la conversación principal).
`hooks`	Hooks que solo se ejecutan mientras este agente está activo.
`memory`	`user`, `project` o `local`: memoria persistente entre sesiones.
`omitClaudeMd`	`true` arranca sin tus CLAUDE.md (más ligero si todo va en la delegación).
`color`	`red`, `blue`, `green`, `yellow`, `purple`, `orange`, `pink`, `cyan`. Un color por agente facilita seguirlos en el panel.

* obligatorio. Lista completa y actualizada: code.claude.com/docs/en/sub-agents

4 Define un formato de salida común

El agregador une mucho mejor si todos los agentes responden con la misma estructura. Inclúyela al final del cuerpo de cada subagente (ya viene en la Plantilla 2):

contrato de salida

```
### RESULTADO: <nombre-del-agente>
- Estado: completo | parcial | bloqueado
- Hallazgos:
  - [alta|media|baja] <hallazgo> — <archivo:línea o fuente>
- Resumen (máx. 5 líneas): <...>
```

Limita la longitud. Cada resultado entra en el contexto de la conversación principal: diez agentes que devuelven informes largos pueden llenarlo.

5 (Opcional) Crea un comando reutilizable

Si vas a repetir el patrón, guarda las instrucciones de orquestador y agregador como una skill en `.claude/skills/paralelo/SKILL.md` (**Plantilla 7**). Después la invocas con `/paralelo <tarea>`.

6 Lanza la ejecución en paralelo

Pide explícitamente que los agentes se lancen **a la vez, en un solo mensaje**. Puedes nombrarlos en lenguaje natural o con @-mención (escribe `@` y elige el agente en la lista). Usa la **Plantilla 5**.

prompt mínimo

```
Lanza en paralelo, en un solo mensaje, @revisor-seguridad,
@revisor-rendimiento y @revisor-tests sobre `git diff main...HEAD`.
Cuando terminen los tres, une sus resultados en un único informe.
```

7 Agrega los resultados

Claude recibe una notificación por cada agente que termina. Cuando tiene todas, redacta la respuesta. Si quieres controlar cómo une (deduplicar, resolver contradicciones, ordenar por prioridad), dale las reglas de la **Plantilla 6** en el mismo prompt de lanzamiento o justo después.

8 Verifica y ajusta

Abre `/tasks` para ver qué modelo usó cada agente, revisa el transcript del que tardó más o devolvió «parcial» y ajusta `description`, `tools`, `model` o `maxTurns`. Los cambios se detectan en segundos sin reiniciar.

05 · EJEMPLO COMPLETO

Revisión de código en paralelo

Tres revisores especializados analizan la misma rama a la vez. La conversación principal une los tres informes.

`.claude/agents/revisor-seguridad.md`

```
---
name: revisor-seguridad
description: Revisa SOLO seguridad en los cambios actuales (secretos, inyección, validación de
entradas, permisos). Úsalo en paralelo con revisor-rendimiento y revisor-tests.
tools: Read, Grep, Glob, Bash
model: sonnet
maxTurns: 25
color: red
---

Eres auditor de seguridad. Solo lectura: no edites archivos.

1. Ejecuta `git diff main...HEAD --stat` y `git diff main...HEAD`.
2. Revisa únicamente seguridad: secretos expuestos, inyección (SQL,
   comandos, rutas), validación de entradas, autenticación y permisos.
3. Ignora estilo, rendimiento y tests: los cubren otros agentes.

### RESULTADO: revisor-seguridad
- Estado: completo | parcial | bloqueado
- Hallazgos:
  - [alta|media|baja] <problema> — <archivo:línea> — <arreglo propuesto>
- Resumen (máx. 5 líneas):
```

`.claude/agents/revisor-rendimiento.md`

```
---
name: revisor-rendimiento
description: Revisa SOLO rendimiento en los cambios actuales (consultas N+1, bucles costosos,
memoria, llamadas de red). Úsalo en paralelo con revisor-seguridad y revisor-tests.
tools: Read, Grep, Glob, Bash
model: haiku
maxTurns: 25
color: yellow
---

Eres especialista en rendimiento. Solo lectura: no edites archivos.
Revisa `git diff main...HEAD` buscando únicamente problemas de rendimiento.
Devuelve el bloque "### RESULTADO: revisor-rendimiento" con el mismo formato
común (estado, hallazgos con prioridad y archivo:línea, resumen).
```

.claude/agents/revisor-tests.md

```
---
name: revisor-tests
description: Ejecuta los tests y revisa la cobertura de los cambios actuales. Úsalo en
paralelo con revisor-seguridad y revisor-rendimiento.
tools: Read, Grep, Glob, Bash
model: sonnet
maxTurns: 30
color: green
---

Eres responsable de calidad. No edites código de producción.
1. Ejecuta la suite de tests del proyecto (<comando, p. ej. mvn test>).
2. Lista los tests que fallan con su error resumido en una línea.
3. Señala qué funciones nuevas o cambiadas en el diff no tienen test.
Devuelve el bloque "### RESULTADO: revisor-tests" con el formato común.
```

Lanzamiento

prompt en Claude Code

```
Revisa mi rama actual con orquestación paralela.
Lanza a la vez, en un solo mensaje, estos tres subagentes:
@revisor-seguridad, @revisor-rendimiento y @revisor-tests.
Cada uno revisa `git diff main...HEAD` desde su especialidad.
Cuando hayan terminado los tres, une sus resultados en un único informe
ordenado por prioridad, sin duplicados y con una tabla de acciones.
```

Qué ocurre

1. Claude (orquestador) hace tres llamadas a la herramienta `Agent` en el mismo turno.
2. Aparecen tres filas en el panel de subagentes, bajo el prompt, cada una con su color.
3. Haiku suele terminar antes; su resultado llega como notificación y Claude espera al resto.
4. Con los tres resultados, Claude (agregador) une, deduplica y ordena.

Resultado agregado (ejemplo)

Prioridad	Hallazgo	Origen	Acción
Alta	Consulta SQL concatenada con entrada del usuario	seguridad	Usar parámetros preparados
Alta	2 tests fallan en `PedidoServiceTest`	tests	Corregir antes del merge
Media	Consulta N+1 al listar pedidos	rendimiento + seguridad	Cargar relaciones en una consulta
Baja	`calcularDescuento()` sin test	tests	Añadir test unitario

Fíjate en la fila «rendimiento + seguridad»: dos agentes detectaron lo mismo y el agregador lo dejó una sola vez anotando ambos orígenes.

06 · MONITORIZACIÓN

Cómo monitorizar la ejecución

Cinco niveles, de lo más inmediato (mirar la terminal) a lo más completo (logs propios con hooks).

Nivel 1 · Panel de subagentes

Debajo del prompt aparece un panel con una fila para la sesión principal y otra por cada subagente en marcha. Si hay subagentes anidados se muestran en árbol, con un contador `(+N)` de descendientes.

Tecla	Acción
`↑` / `↓`	Moverte entre filas.
`Enter`	Abrir el transcript del agente y enviarle mensajes de seguimiento (corregirle a mitad de tarea).
`x`	Detener el agente seleccionado, o descartar su fila si ya terminó.
`Esc`	Volver al prompt.

Cuando un agente termina bien, su fila desaparece y el pie muestra durante 30 segundos `/tasks to see subagents`. Si falla o lo detienes, la fila se mantiene 30 segundos.

Nivel 2 · `/tasks`

- Lista el trabajo en segundo plano: los subagentes en marcha arriba, los terminados marcados como hechos.
- Muestra el **modelo** de cada subagente y su nivel de **effort** si lo tiene. Útil para confirmar que tu `model: haiku` se aplicó.
- Selecciona uno y pulsa `Enter` para abrir su transcript.

Nivel 3 · Permisos y control en vivo

- Los avisos de permiso de un subagente aparecen en tu sesión con su nombre. Aprobar con una opción «para toda la sesión» se aplica también a la conversación principal.
- Si Claude está esperando a un agente, `Ctrl+B` lo pasa a segundo plano para que sigas trabajando.
- Para corregir a un agente sin pararlo: ábrelo con `Enter` y escríbele. Con `Ctrl+Enter` lee tu mensaje antes de terminar lo que esté haciendo.

Nivel 4 · Transcripts en disco

Cada subagente guarda su conversación completa en un archivo propio, independiente de la conversación principal (no se pierde aunque esta se compacte):

ubicación

```
~/.claude/projects/<proyecto>/<sessionId>/subagents/
    agent-<agentId>.jsonl        ← conversación completa del subagente
    agent-<agentId>.meta.json    ← metadatos (nombre, tipo…)
```

- Son JSONL: una línea por mensaje o evento. Puedes leerlos con `jq` o abrirlos en el editor.
- Si un subagente compactó su contexto verás una línea con `"subtype": "compact_boundary"` y `preTokens` (tokens usados antes de compactar). Si aparece a menudo, ese agente tiene demasiado trabajo: divídelo.
- Se borran automáticamente tras `cleanupPeriodDays` (30 días por defecto).

Nivel 5 · Hooks SubagentStart y SubagentStop

Los hooks ejecutan un script cuando ocurre un evento. `SubagentStart` salta cuando arranca un subagente y `SubagentStop` cuando termina. Con ellos puedes registrar cuándo empieza y acaba cada agente, cuánto tarda y cuántos se solapan. El script recibe por stdin un JSON con, entre otros, `session_id`, `agent_id`, `agent_type` (el `name` del agente) y, al terminar, `agent_transcript_path`.

Configúralos en `.claude/settings.json` (**Plantilla 8**). Con `matcher` puedes limitarlos a un agente concreto: `"matcher": "revisor-seguridad"`.

.claude/hooks/log-subagentes.sh

```
#!/bin/bash
# .claude/hooks/log-subagentes.sh
# Registra cada inicio y fin de subagente en .claude/logs/subagentes.jsonl
# Requiere jq.  chmod +x .claude/hooks/log-subagentes.sh

EVENTO="$1"                          # start | stop
INPUT=$(cat)                         # JSON que Claude Code envía por stdin
mkdir -p .claude/logs

echo "$INPUT" | jq -c --arg ev "$EVENTO" --arg ts "$(date +%s)" '{
  ts:          ($ts | tonumber),
  evento:      $ev,
  sesion:      .session_id,
  agente_id:   .agent_id,
  tipo:        .agent_type,
  transcript:  (.agent_transcript_path // null)
}' >> .claude/logs/subagentes.jsonl

exit 0                               # nunca bloquea al agente
```

Tras una ejecución, `.claude/logs/subagentes.jsonl` tendrá dos líneas por agente. El script de resumen de la Plantilla 8 calcula la duración de cada uno: si los `inicio` son casi iguales, se lanzaron de verdad en paralelo.

07 · PROBLEMAS

Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
Claude no encuentra el subagente nuevo	La carpeta `agents` no existía al abrir la sesión	Reinicia Claude Code. Después, los cambios se detectan solos.
Un campo del frontmatter no tiene efecto	Nombre mal escrito (no es camelCase)	`claude --debug` y busca «does not recognize or apply».
El archivo se ignora por completo	Falta `name` o `description`, YAML inválido, o `---` no está en la primera línea	`claude plugin validate .claude/agents`.
«Agent would be spawned with zero tools»	Ninguna entrada de `tools` corresponde a una herramienta disponible	Revisa la ortografía y que la herramienta exista en segundo plano.
Claude ejecuta los agentes uno detrás de otro	El prompt no pide paralelo, o la tarea parece dependiente	Pide «en paralelo, en un solo mensaje» y deja claro que son independientes.
«Concurrent subagent limit reached»	Más de 20 agentes en marcha a la vez	Agrupa subtareas o sube `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`.
Los agentes se pisan cambios	Varios escriben en los mismos archivos	`isolation: worktree` o agentes de solo lectura.
La conversación principal se queda sin contexto	Resultados demasiado largos	Limita la salida en el contrato (palabras, nº de hallazgos).
Un resultado llega marcado como parcial	Alcanzó `maxTurns`	Pide a Claude que lo reanude (usa `SendMessage` con su ID) o sube el tope.
Un agente falla por error de API	Límite de uso o error del servidor	Su última salida no se pierde. Cuando se resuelva, pide reintentar o reanudar.
El agente no conoce el contexto de la conversación	Los subagentes no ven el historial	Pon todo lo necesario en la delegación o usa `/subtask` (fork con historial).
No puedes reanudar Explore o Plan	Son de un solo uso y no devuelven ID	Usa `general-purpose` o un subagente propio.

DIAGNÓSTICO DE CONFIGURACIÓN

- `claude --debug`: el log en `~/.claude/debug/<session-id>.txt` indica campos del frontmatter mal escritos («does not recognize or apply») y archivos que no cargaron.
- `claude agents`: lista los subagentes configurados sin abrir sesión.
- `claude plugin validate .claude/agents`: detecta frontmatter YAML que no se puede leer.
- `/doctor`: avisa de agentes con el mismo `name` en una misma carpeta.

08 · CRITERIO

Buenas prácticas y costes

Diseño

- **Un agente, una responsabilidad.** «Revisa seguridad» funciona mejor que «revisa todo».
- **Descripciones que no se solapen.** El orquestador elige por la `description`; si dos se parecen, delegará mal.
- **Entre 2 y 6 agentes por ejecución.** Más agentes significa más resultados que unir y más contexto ocupado.
- **Contrato de salida común y corto** en todos los agentes.
- **Delegaciones completas.** Rutas, criterios, formato y lo que NO debe tocar.
- **Versiona `.claude/agents/` en git** para reutilizar y mejorar los agentes.

Seguridad

- Mínimo de herramientas: si solo analiza, `Read, Grep, Glob`.
- Para bloquear comandos concretos sin quitar Bash, usa reglas en `permissions.deny` (por ejemplo `Bash(git push *)`) o un hook `PreToolUse`.
- Para impedir que un agente lance otros, quita `Agent` de sus `tools`.
- Claude Code revisa el informe final de cada subagente y marca texto que imita instrucciones del sistema. No sustituye a limitar herramientas.

Costes y límites de uso

- Cada subagente consume de los mismos límites de uso que tu conversación. Cinco agentes en paralelo gastan cinco veces más **en ese momento**.
- Asigna modelos según la dificultad: Haiku para búsquedas y tareas mecánicas, Sonnet para análisis, Opus solo donde haga falta razonamiento profundo.
- Para forzar un modelo en todos los subagentes: `CLAUDE_CODE_SUBAGENT_MODEL` junto con `CLAUDE_CODE_SUBAGENT_MODEL_FORCE=1` en el bloque `env` de `settings.json`.
- `omitClaudeMd: true` ahorra contexto en agentes que no necesitan tus reglas de proyecto.

PARALELA CON OTROS PATRONES

En proyectos reales se combinan: un paso secuencial prepara los datos, un bloque paralelo los analiza y otro paso secuencial redacta el informe. Un subagente también puede lanzar sus propios subagentes en paralelo (hasta 3 niveles por defecto), por ejemplo un revisor que lanza un verificador por cada hallazgo.

PARTE 09

Plantillas

Diez plantillas para copiar en tu proyecto o rellenar a mano. Sustituye todo lo que aparece entre < >.

PLANTILLA 1

Canvas de diseño de la división

Rellénalo antes de crear ningún agente. Si una subtarea tiene algo en la columna «Depende de», no la pongas en el mismo bloque paralelo.

Dónde va: papel o Notion, antes de empezar

Tarea completa

Resultado final esperado	Formato de la respuesta agregada

#	Subtarea	Agente	Archivos / fuente	Herramientas	Modelo	Depende de
1						
2						
3						
4						
5						
6						

Reglas de agregación (duplicados, conflictos, orden)	¿Algún agente escribe archivos? → ¿worktree?

Test de independencia

- [ ] Cada subtarea se explica sin el resultado de otra
- [ ] Ningún par de agentes escribe en los mismos archivos
- [ ] El orden de llegada de resultados da igual
- [ ] Hay un formato de salida común

PLANTILLA 2

Subagente genérico

Base para cualquier agente del bloque paralelo. Incluye el contrato de salida común.

Dónde va: .claude/agents/<nombre>.md

.claude/agents/nombre-del-agente.md

```
---
name: nombre-del-agente
description: Qué hace en una frase y cuándo usarlo. Use proactively when <situación>.
tools: Read, Grep, Glob          # solo lo que necesita
model: sonnet                    # haiku | sonnet | opus | inherit
maxTurns: 25                     # tope de seguridad
color: orange                    # color en el panel y en /tasks
---

Eres <rol especializado>. Tu única responsabilidad es <subtarea>.

## Alcance
- Trabaja SOLO sobre: <archivos, carpeta, módulo o pregunta>.
- No toques: <lo que pertenece a otros agentes>.

## Proceso
1. <primer paso>
2. <segundo paso>
3. <tercer paso>

## Formato de salida (obligatorio)
Devuelve exactamente este bloque, sin texto antes ni después:

### RESULTADO: <nombre-del-agente>
- Estado: completo | parcial | bloqueado
- Hallazgos:
  - [prioridad alta|media|baja] <hallazgo> — <archivo:línea o fuente>
- Resumen (máx. 5 líneas): <...>
- Dudas o dependencias detectadas: <ninguna | ...>
```

PLANTILLA 3

Subagente investigador (solo lectura)

Para lanzar varias copias en paralelo, una por módulo o tema. Usa Haiku para que sea barato.

Dónde va: .claude/agents/investigador-modulo.md

.claude/agents/investigador-modulo.md

```
---
name: investigador-modulo
description: Investiga un único módulo o tema y devuelve un resumen estructurado. Úsalo en
paralelo, un agente por módulo.
tools: Read, Grep, Glob, WebFetch, WebSearch
model: haiku
maxTurns: 20
color: cyan
---

Eres un investigador de solo lectura. Recibirás UN módulo o tema concreto
en el mensaje de delegación. No modifiques ningún archivo.

1. Localiza los archivos o fuentes relevantes del tema asignado.
2. Lee lo imprescindible. No pegues archivos completos en tu respuesta.
3. Responde con el formato de salida común (máx. 300 palabras).

### RESULTADO: investigador-modulo — <tema>
- Estado: completo | parcial | bloqueado
- Qué hace: <...>
- Puntos clave (máx. 5): <...>
- Riesgos o dudas: <...>
- Archivos o fuentes consultadas: <lista>
```

PLANTILLA 4

Subagente que escribe en un worktree aislado

Para cambios en paralelo sobre el mismo repositorio sin conflictos: cada agente trabaja en su propia copia.

Dónde va: .claude/agents/migrador-carpeta.md

.claude/agents/migrador-carpeta.md

```
---
name: migrador-carpeta
description: Aplica una migración mecánica a UNA carpeta concreta en un worktree aislado.
Úsalo en paralelo, un agente por carpeta.
tools: Read, Edit, Write, Grep, Glob, Bash
model: sonnet
isolation: worktree            # copia aislada del repo; se limpia si no hay cambios
permissionMode: acceptEdits
maxTurns: 40
color: green
---

Migras SOLO la carpeta indicada en el mensaje de delegación.
No edites archivos fuera de ella.

1. Lista los archivos de la carpeta asignada.
2. Aplica la regla de migración: <describe la regla>.
3. Ejecuta los tests de esa carpeta: <comando>.
4. Haz commit en tu worktree con el mensaje "migración: <carpeta>".

### RESULTADO: migrador-carpeta — <carpeta>
- Estado: completo | parcial | bloqueado
- Archivos cambiados: <n> · Tests: pasan | fallan (<detalle>)
- Rama/worktree: <nombre>
- Revisar a mano: <lista o "nada">
```

PLANTILLA 5

Prompt de lanzamiento en paralelo

Lo escribes en Claude Code. Fuerza el lanzamiento simultáneo y le recuerda al orquestador que pase el contexto completo.

Dónde va: el prompt de Claude Code

prompt

```
Quiero resolver esta tarea con orquestación paralela:
<describe la tarea completa>

Divídela en estas subtareas independientes y lanza un subagente por cada una
EN PARALELO, en un solo mensaje y sin esperar a que termine ninguno
para lanzar el siguiente:

1. @agente-A → <subtarea 1, con su alcance exacto>
2. @agente-B → <subtarea 2, con su alcance exacto>
3. @agente-C → <subtarea 3, con su alcance exacto>

Reglas para ti como orquestador:
- No hagas tú ninguna de las subtareas mientras los agentes trabajan.
- Pasa a cada agente todo el contexto que necesite: no ve esta conversación.
- Cuando TODOS hayan terminado, actúa como agregador (instrucciones abajo).
```

PLANTILLA 6

Prompt de agregación

Reglas para que la conversación principal una bien los resultados. Pégalo al final del prompt de lanzamiento o cuando lleguen los resultados.

Dónde va: el prompt de Claude Code

prompt

```
Ahora actúa como agregador. Ya tienes los resultados de todos los agentes.

1. Comprueba que han llegado los N resultados. Si alguno es "parcial"
   o "bloqueado", dilo al principio y no lo des por completo.
2. Une los hallazgos en una sola respuesta:
   - Elimina duplicados (mismo problema detectado por varios agentes:
     mantenlo una vez y anota quién lo detectó).
   - Si dos agentes se contradicen, muestra ambas posturas y propón
     cómo resolverlo.
   - Ordena por prioridad: alta → media → baja.
3. Formato final:
   ## Resumen (3-5 líneas)
   ## Hallazgos por prioridad (tabla: prioridad | hallazgo | origen | acción)
   ## Conflictos entre agentes
   ## Siguientes pasos
```

PLANTILLA 7

Skill /paralelo (orquestador + agregador reutilizable)

Guarda el patrón completo como comando. Uso: /paralelo revisa la rama actual con todos los revisores.

Dónde va: .claude/skills/paralelo/SKILL.md

.claude/skills/paralelo/SKILL.md

```
---
name: paralelo
description: Ejecuta una tarea con orquestación paralela usando los subagentes de
.claude/agents. Úsalo cuando el usuario escriba /paralelo seguido de la tarea.
---

# Orquestación paralela

Tarea del usuario: $ARGUMENTS

## Fase 1 · Orquestador
1. Divide la tarea en 2-6 subtareas que NO dependan unas de otras.
   Si alguna depende de otra, avisa y propone hacerla después.
2. Asigna a cada subtarea el subagente más adecuado de .claude/agents.
3. Muestra la división en una tabla (subtarea | agente | alcance).
4. Lanza todos los subagentes en el mismo turno, en paralelo.
   Cada delegación incluye el contexto completo que el agente necesita.

## Fase 2 · Espera
- No adelantes conclusiones mientras haya agentes en marcha.

## Fase 3 · Agregador
- Verifica que han llegado todos los resultados.
- Une, deduplica, marca conflictos y ordena por prioridad.
- Termina con "Siguientes pasos".
```

PLANTILLA 8

Monitorización con hooks

Registra inicio y fin de cada subagente en un JSONL y calcula duraciones. Requiere jq. Da permisos de ejecución a los scripts con chmod +x.

Dónde va: .claude/settings.json + .claude/hooks/

.claude/settings.json

```
{
  "env": {
    "CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS": "6"
  },
  "hooks": {
    "SubagentStart": [
      {
        "hooks": [
          { "type": "command",
            "command": ".claude/hooks/log-subagentes.sh start" }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command",
            "command": ".claude/hooks/log-subagentes.sh stop" }
        ]
      }
    ]
  }
}
```

El script `log-subagentes.sh` está en la Parte 06. Este es el de resumen:

.claude/hooks/resumen-subagentes.sh

```
#!/bin/bash
# .claude/hooks/resumen-subagentes.sh
# Muestra duración de cada subagente y cuántos se solaparon en paralelo.

jq -s '
  group_by(.agente_id)
  | map(select(length == 2))
  | map({
      tipo:     .[0].tipo,
      id:       .[0].agente_id,
      inicio:   (map(select(.evento=="start"))[0].ts),
      fin:      (map(select(.evento=="stop"))[0].ts)
    } | . + {segundos: (.fin - .inicio)})
  | sort_by(.inicio)
' .claude/logs/subagentes.jsonl
```

salida de ejemplo

```
[
  { "tipo": "revisor-seguridad",   "inicio": 1761424201, "fin": 1761424498, "segundos": 297 },
  { "tipo": "revisor-rendimiento", "inicio": 1761424201, "fin": 1761424376, "segundos": 175 },
  { "tipo": "revisor-tests",       "inicio": 1761424202, "fin": 1761424560, "segundos": 358 }
]
```

Los tres `inicio` difieren en un segundo: se lanzaron en paralelo. El tiempo total lo marca el más lento (358 s).

PLANTILLA 9

Subagentes temporales con --agents

Para probar una configuración sin crear archivos. Solo existen durante esa sesión. En PowerShell usa @' ... '@ para el JSON.

Dónde va: terminal

terminal (macOS, Linux, WSL)

```
claude --agents '{
  "seguridad": {
    "description": "Revisa seguridad. Úsalo en paralelo con rendimiento y tests.",
    "prompt": "Eres auditor de seguridad. Solo lectura. Devuelve el formato RESULTADO.",
    "tools": ["Read", "Grep", "Glob"],
    "model": "sonnet"
  },
  "rendimiento": {
    "description": "Revisa rendimiento. Úsalo en paralelo con seguridad y tests.",
    "prompt": "Eres experto en rendimiento. Solo lectura. Devuelve el formato RESULTADO.",
    "tools": ["Read", "Grep", "Glob"],
    "model": "haiku"
  }
}'
```

PLANTILLA 10

Checklist antes de lanzar

Repásala antes de cada ejecución importante.

Dónde va: papel o Notion

- [ ] Las subtareas pasan el test de independencia
- [ ] Cada agente tiene `name` y `description` únicos y concretos
- [ ] Herramientas mínimas en cada agente (solo lectura si no escribe)
- [ ] Agentes que escriben → `isolation: worktree`
- [ ] Modelo asignado según dificultad (Haiku / Sonnet / Opus)
- [ ] `maxTurns` definido para que nadie retrase la agregación
- [ ] Contrato de salida común y con longitud limitada
- [ ] El prompt pide lanzar «en paralelo, en un solo mensaje»
- [ ] Reglas de agregación escritas (duplicados, conflictos, orden)
- [ ] Hooks de log activos si quieres medir tiempos
- [ ] Número de agentes por debajo del límite de concurrencia

Notas

10 · REFERENCIA RÁPIDA

Chuleta

EL PATRÓN

Usuario → Orquestador divide → N agentes a la vez → Agregador espera a todos y une → Usuario.
Condición: subtareas independientes.

EN CLAUDE CODE

Orquestador y agregador = **conversación principal**.
Agentes = **subagentes** en `.claude/agents/*.md`.

Quiero…	Uso
Crear un agente de proyecto	`.claude/agents/nombre.md` con `name` + `description`
Crear un agente para todos mis proyectos	`~/.claude/agents/nombre.md`
Probar agentes sin crear archivos	`claude --agents '{…}'`
Lanzar en paralelo	«Lanza en paralelo, en un solo mensaje, @a, @b y @c…»
Forzar un agente concreto	`@"nombre (agent)"` o `@agent-nombre`
Agente de solo lectura	`tools: Read, Grep, Glob`
Que no se pisen al escribir	`isolation: worktree`
Abaratar	`model: haiku` · `omitClaudeMd: true`
Que no se alargue	`maxTurns: 25`
Ver qué se está ejecutando	Panel bajo el prompt · `/tasks`
Ver qué hizo un agente	`Enter` en su fila · `~/.claude/projects/…/subagents/`
Detener un agente	`x` en su fila
Corregirlo a mitad de tarea	`Enter` en su fila y escribirle
Pasar una espera a segundo plano	`Ctrl+B`
Registrar inicio y fin	Hooks `SubagentStart` / `SubagentStop`
Detectar campos mal escritos	`claude --debug`
Limitar agentes simultáneos	`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` (20)
Limitar anidación	`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` (3)
Bloquear un agente	`"permissions": {"deny": ["Agent(nombre)"]}`
Reanudar un agente que quedó parcial	Pedírselo a Claude (usa `SendMessage` con su ID)

Fuentes

Claude Code Docs — Create custom subagents: code.claude.com/docs/en/sub-agents
Claude Code Docs — Hooks (SubagentStart, SubagentStop): code.claude.com/docs/en/hooks
Claude Agent SDK — Hooks: platform.claude.com/docs/en/agent-sdk/hooks
Consultadas el 25 de octubre de 2026. Claude Code se actualiza con frecuencia: si algo no coincide, revisa la documentación.